Neuromorphic Cochlea and SNN-Based Pitch Classification

From Preprocessing to “Absolute Pitch” Models

Project Goals and Timeline

Research Objective

Develop a bio-inspired spiking neural network for robust musical pitch classification.

Experimental Timeline:

1. Implemented custom auditory preprocessing pipeline
2. Discovered and fixed ERB scaling ($\times$1000 error)
3. 3-tone classification (C4, A4, A5) – 100% accuracy
4. Full 88-key piano classification – 58% accuracy
5. STDP-based unsupervised learning comparison

Key Finding: Handy preprocessing shows superior noise robustness despite slower training convergence.

ERB Scale: Three Implementations

Equivalent Rectangular Bandwidth (ERB) Scale

Perceptual frequency scale modeling human auditory filter bandwidths.

Linear ERB (Glasberg & Moore, 1990):

$$\mathrm{ERB}(F) = 24.7 \cdot (4.37F + 1) \quad [\mathrm{Hz}]$$

$$\mathrm{ERBS}(f) = 21.4 \log_{10}(1 + 4.37f) \quad [\mathrm{kHz}]$$

Quadratic ERB:

$$\mathrm{ERB}(F) = 6.23F^2 + 93.39F + 28.52$$

$$\mathrm{ERBS}(f) = 11.17 \cdot \ln\left(\frac{f + 0.312}{f + 14.675}\right) + 43.0$$

Voicebox ERB:

$$\mathrm{ERBS}(f) = a \ln\left(h - \frac{k}{f + c}\right)$$

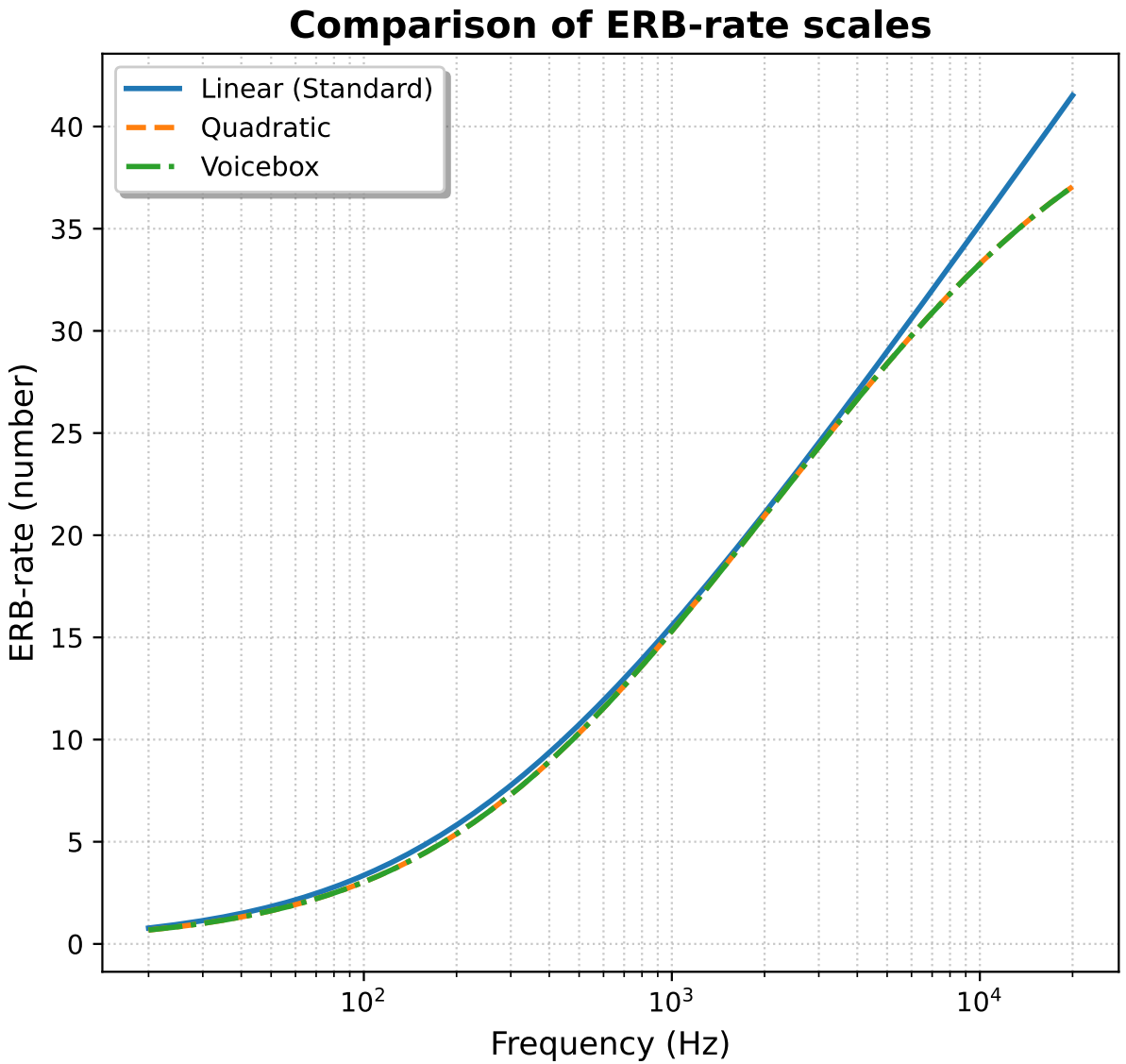
where $a = 11.17$, $h = 47.07$, $k = 676170$, $c = 14678$

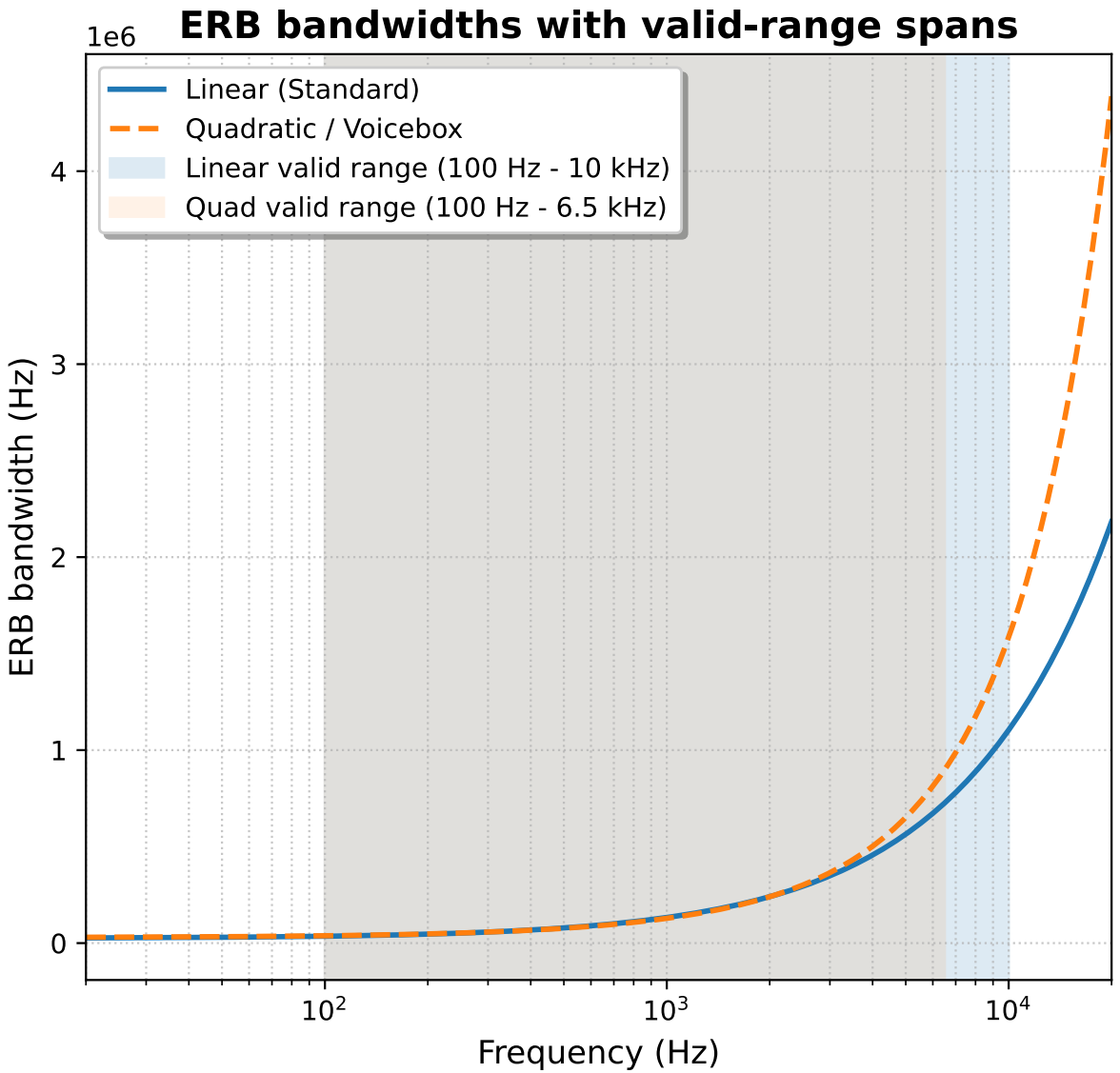
Inverse Mapping:

$$f = \frac{k}{h - \exp(\mathrm{ERBS}/a)} - c$$

Critical Bug: Quadratic and Linear ERB had wrong scaling factor ($\times 1000$) at return!

ERB Scale: Valid ranges





Gammatone Filterbank

Filter Impulse Response

Time-domain FIR approximation of cochlear filtering:

$$h_i(t) = t^{n-1} e^{-2\pi b_i t} \cos(2\pi f_i t + \phi), \quad n = 4 \tag{1}$$

where f_i are center frequencies spaced on ERB scale, $b_i = 1.019 \cdot \mathrm{ERB}(f_i)$

Filterbank Processing:

$$x_i(t) = (x * h_i)(t), \quad i = 1, \ldots, N_{\text{chan}} \tag{2}$$

Parameters:

- $N_{\text{chan}} = 50$ channels
- $f_{\text{low}} = 100$ Hz
- $f_{\text{high}} = 6500$ Hz
- IR duration: 50 ms

Implementation:

- FFT convolution for speed
- Normalized impulse response
- Mode: 'same' for alignment

Inner Hair Cell and Spike Generation

Inner Hair Cell (IHC) Model

Soft half-wave rectification followed by power-law compression:

$$r_i(t) = \max(0, x_i(t))^{0.7} \tag{3}$$

Poisson Spike Generation

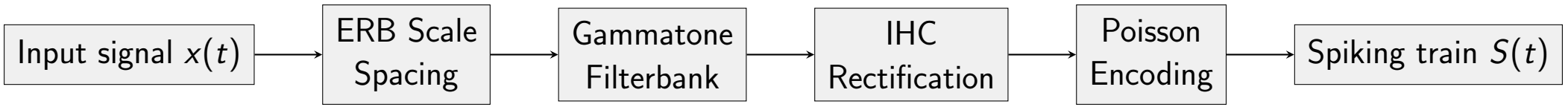
Inhomogeneous Poisson process with rate proportional to IHC output:

$$P(\text{spike in } [t, t+dt)) = \lambda_i(t) \cdot dt = \gamma \cdot \frac{r_i(t)}{r_{\max}} \cdot \max_{\text{rate}} \cdot dt \tag{4}$$

Relative Rate Logic:

- Global maximum across all channels: $r_{\max} = \max_{i,t} r_i(t)$
- Preserves relative loudness between channels
- High-energy channels fire at $\max_{\text{rate}} = 800$ Hz, low-energy near 0 Hz

Pipeline Architecture



Two Implementations Compared:

"Handy" (Custom):

- Custom ERB implementations
- Manual gammatone filters
- Relative threshold Poisson
- Global/max normalization

"Spikify" (Library):

- Built-in ERB spacing
- Optimized filterbank
- Standard Poisson encoder
- Channel-wise processing

The ERB Scaling Bug and Fix

Bug Discovery

Quadratic and Linear ERB scales had incorrect scaling at return: `return value * 1000` instead of correct frequency in Hz.

Impact on Spike Generation:

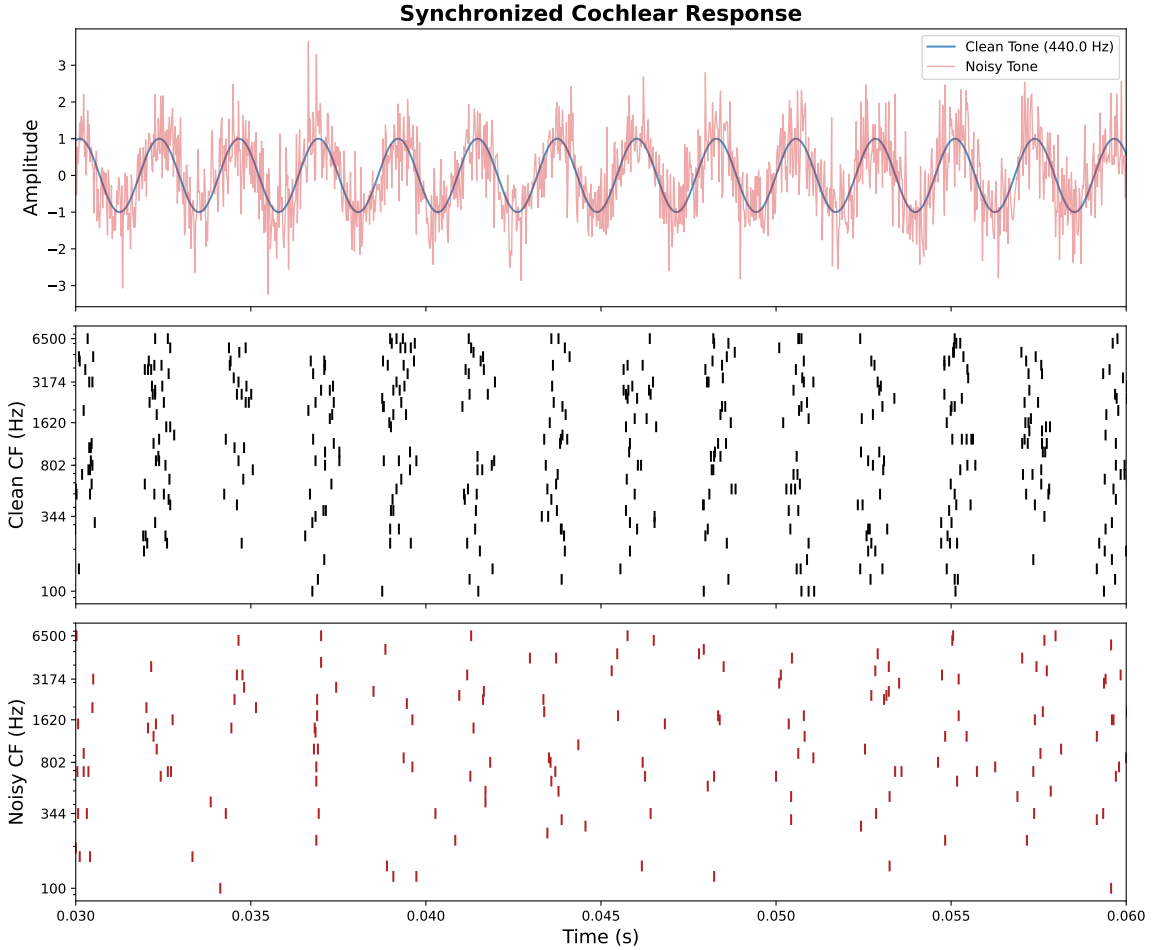
- Center frequencies computed incorrectly
- Filterbank channels misaligned with input tones
- Spike patterns had poor phase-locking
- Classification accuracy degraded significantly

After Fix:

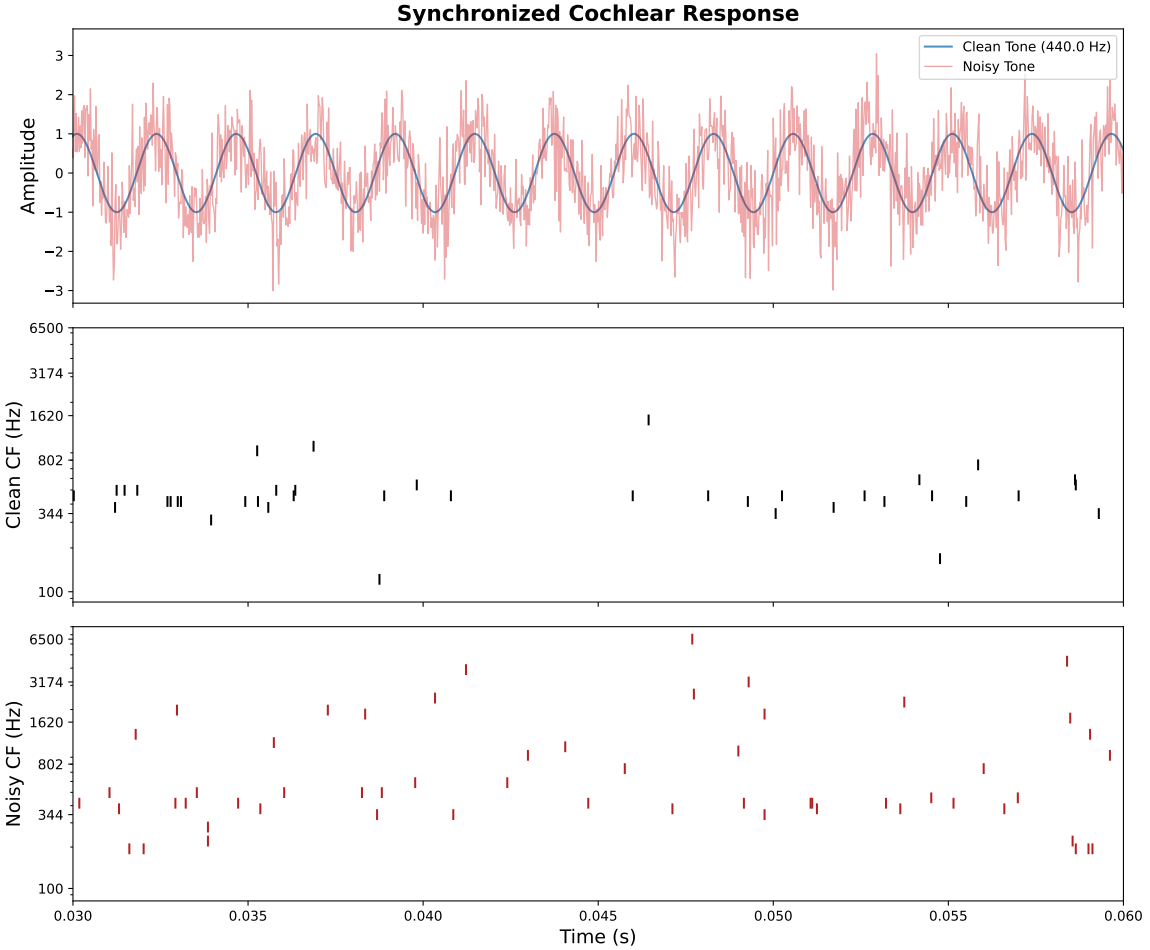
- Correct center frequency placement
- Improved phase-locking to stimulus period
- Clearer frequency-specific channel activation
- 100% accuracy on 3-tone classification

The ERB Scaling Bug and Fix

Handy before

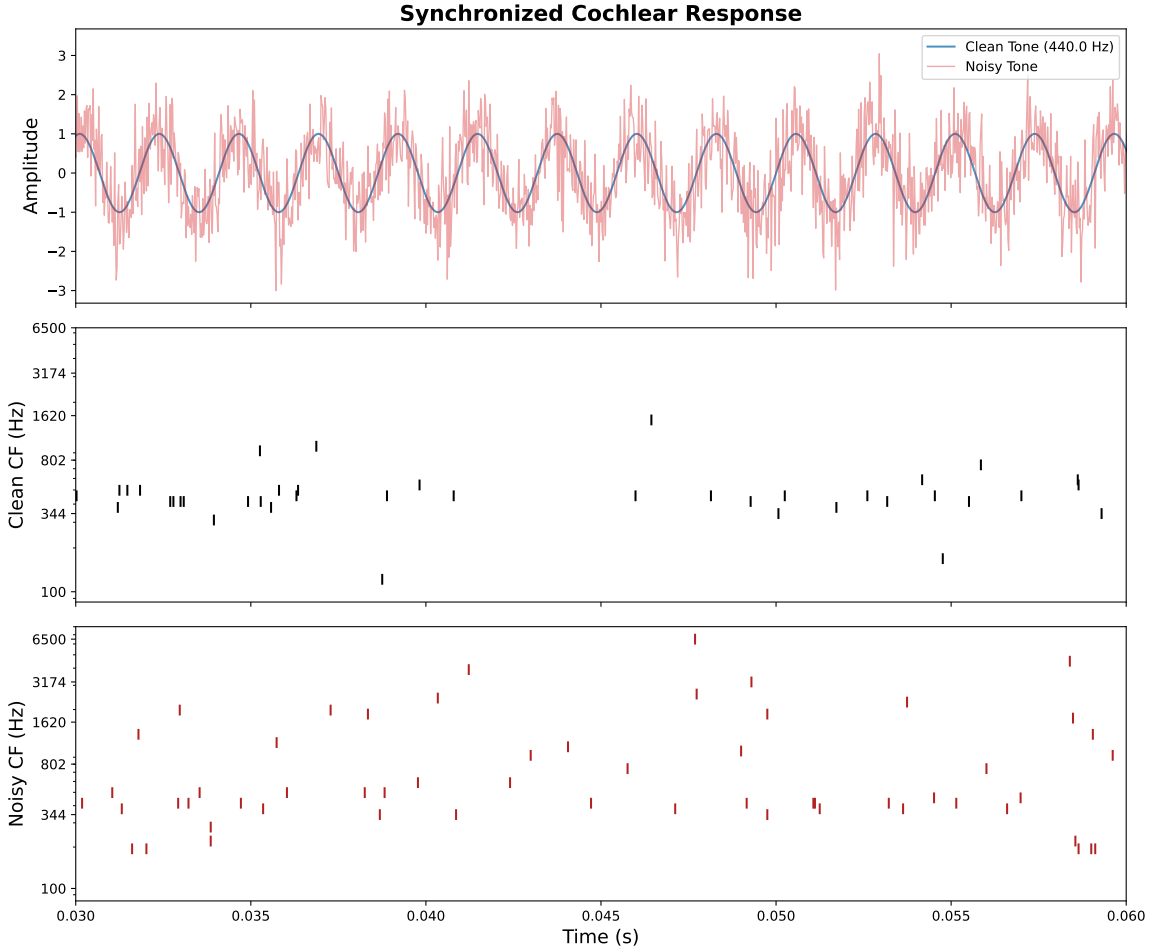


Handy after

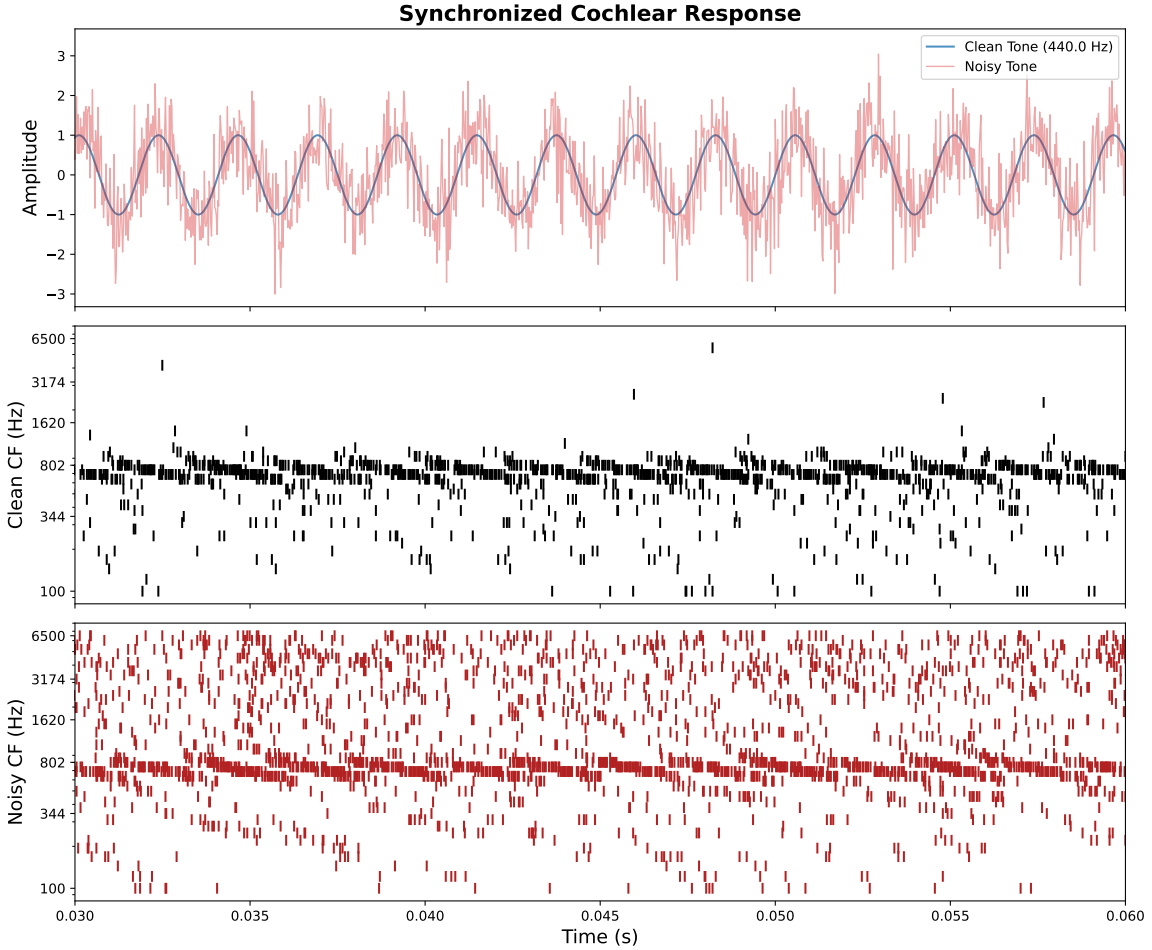


The ERB Scaling Handy vs Spikify

Handy



Spikify



SNN Architecture: LIF Neuron Dynamics

Leaky Integrate-and-Fire (LIF) Model

$$\tau_m \frac{dV_j}{dt} = -(V_j - V_{\text{rest}}) + R \sum_i w_{ij} s_i(t) \tag{5}$$

$$V_j \geq V_{\text{th}} \Rightarrow \text{spike } s_j = 1, \quad V_j \leftarrow V_{\text{reset}} \tag{6}$$

Supervised SNN Architecture (PitchSNN):

- **Input:** 50 channels (cochlear filters); **Output:** 3 or 88 LIF neurons;
- **Hidden:** 100 LIF neurons ($\beta = 0.9$)
- **Surrogate gradient:** Fast sigmoid (slope $= 25$)

Training: Cross-entropy loss on time-averaged membrane potentials:

$$\mathcal{L} = \text{CE}\left(\frac{1}{T} \sum_{t=1}^{T} \mathbf{V}_{\text{out}}(t),\ y_{\text{target}} \right) \tag{7}$$

3 notes Dataset and Experimental Setup

Dataset Generation:

- **Tones:** C4 (261.63 Hz), A4 (440 Hz), A5 (880 Hz)
- **Duration:** 100 ms pure sine waves
- **Sampling rate:** 44.1 kHz
- **Samples per class:** 60 (training)
- **SNR variance:** Random 10-30 dB for robustness

Model Configuration:

- Input: 50 cochlear channels
- Hidden: 100 LIF neurons
- Output: 3 classes
- Optimizer: Adam (lr = 0.005); Epochs: 20; Batch size: 16

Result

100% training accuracy achieved after preprocessing fix for both handy and spikify.

Noise Robustness Analysis

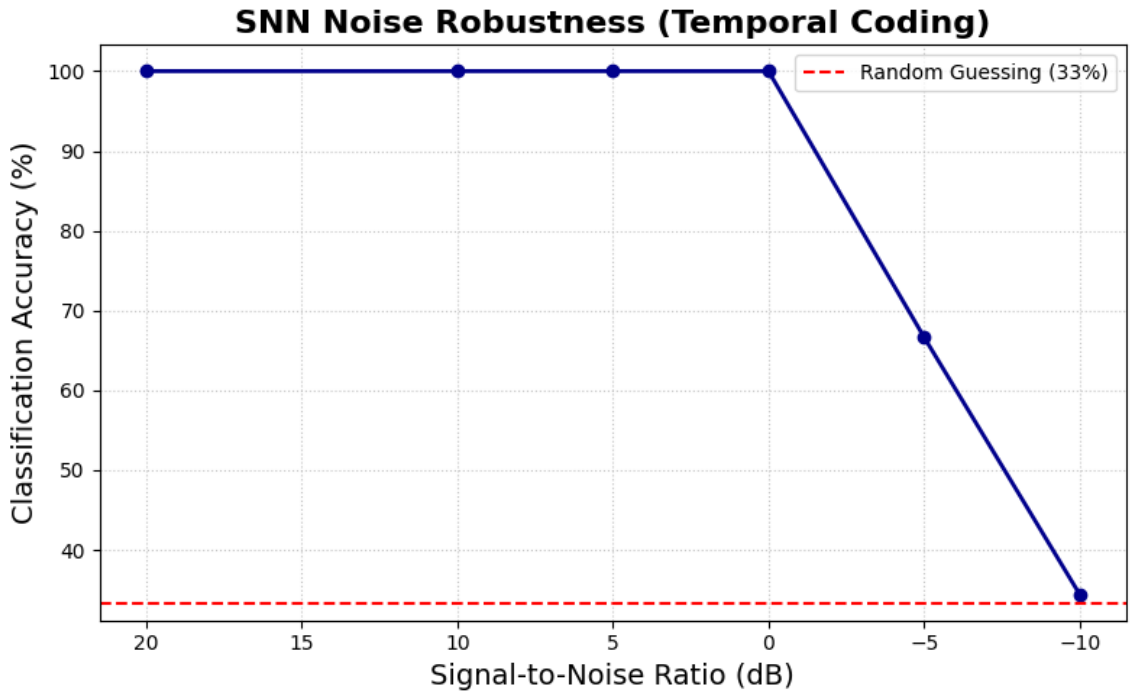
Evaluation Protocol

Test accuracy at progressively worse SNR levels: 20, 10, 5, 0, -5, -10 dB

Key Finding:

- **Handy preprocessing:** Stable training, maintains performance at negative dB
- **Spikify preprocessing:** Faster training accuracy increase, unstable at low SNR

Spikify noise robustness



Full Piano Dataset Generation

Piano Range: A0 (27.5 Hz) to C8 (4186.01 Hz)

Frequency Calculation:

$$f_k = f_{A0} \cdot 2^{k/12} = 27.5 \cdot 2^{k/12}, \quad k = 0, 1, \ldots, 87 \tag{8}$$

Dataset Parameters:

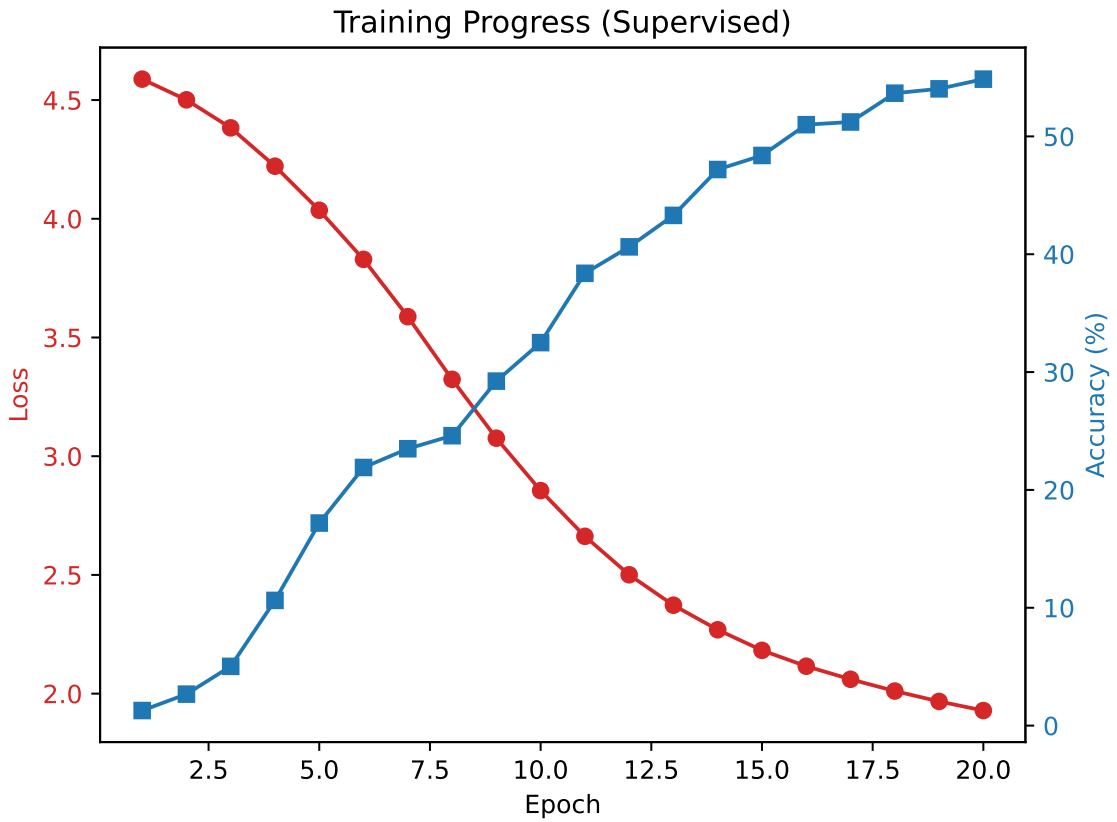
- 88 classes (full piano keyboard)
- 50 samples per class
- Train/test split: 80/20
- Total samples: 4,400
- Temporal binning: 1 ms (100 time steps)

Adaptive Filterbank: Automatically adjusts f_{low} and f_{high} to cover full piano range:

$$f_{\text{low}} = \min(100, 27.5), \quad f_{\text{high}} = \max(6500, 4186) \tag{9}$$

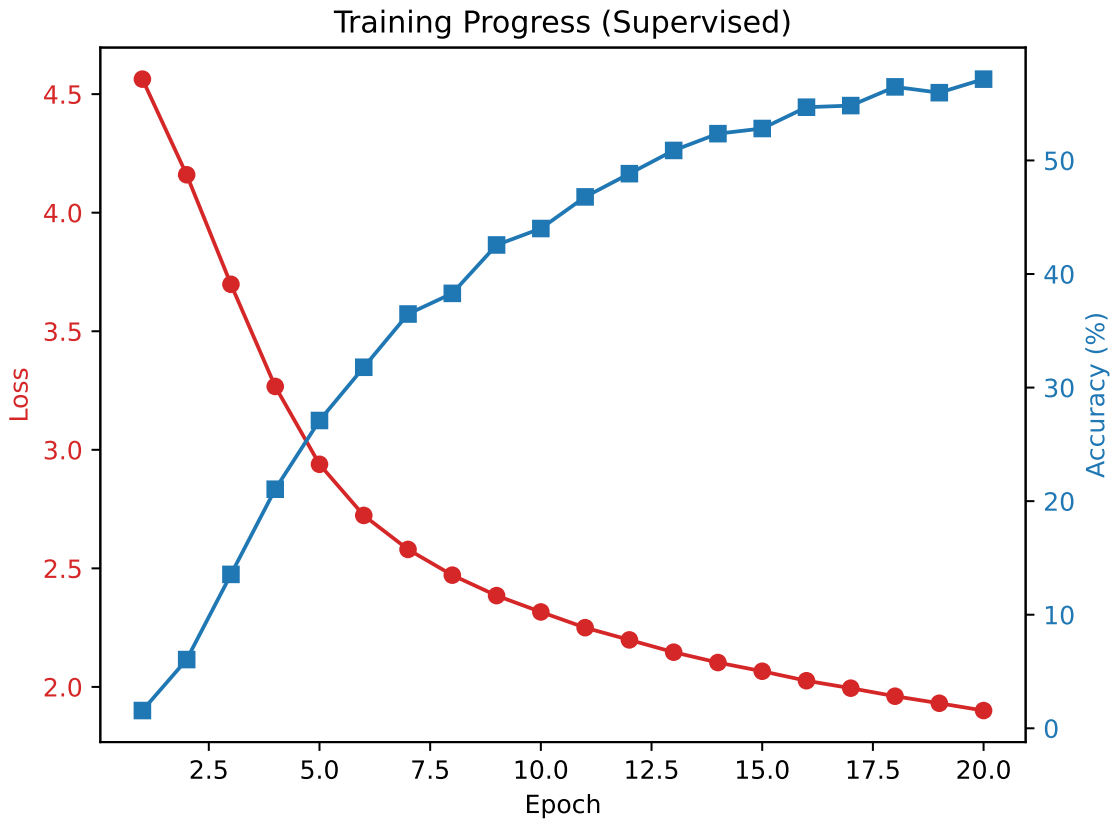
88-Key Piano Training Results: Accuracy Curves

Handy Preprocessing



- Slower convergence
- More stable training
- Final: ∼58% accuracy

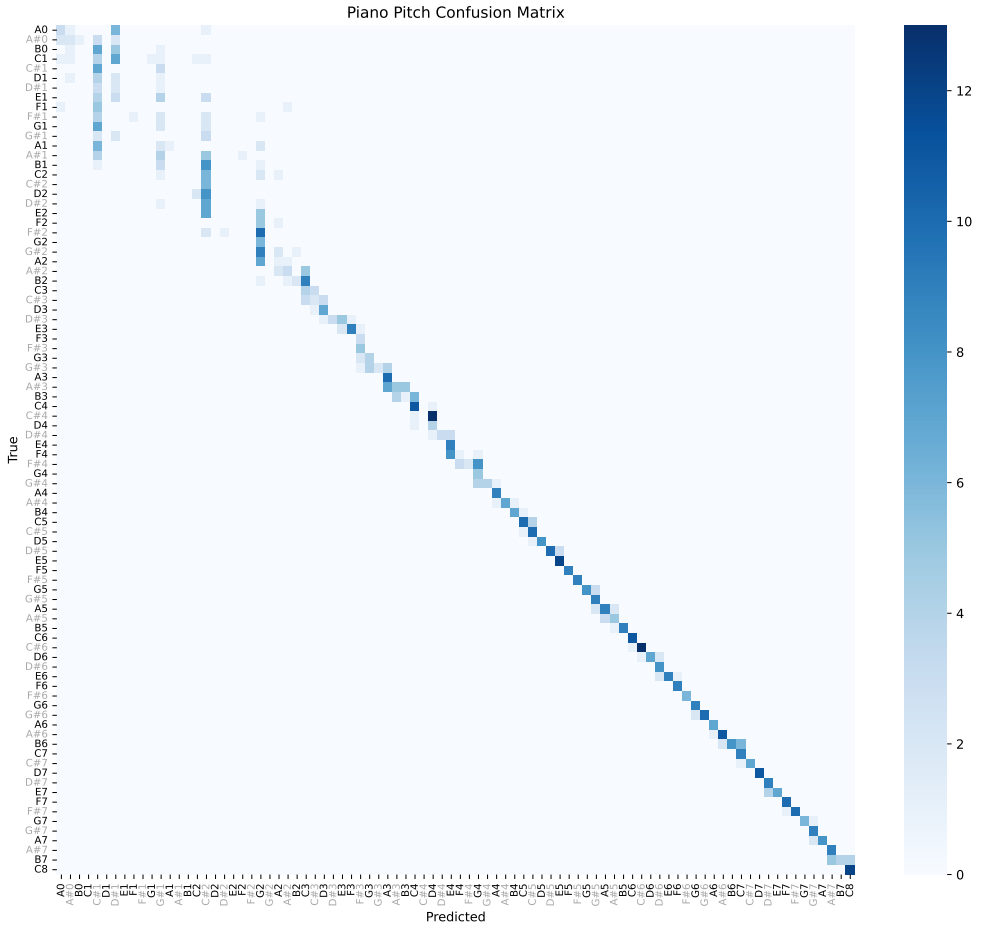
Spikify Preprocessing



- Faster initial learning
- Higher variance
- Final: ∼58% accuracy

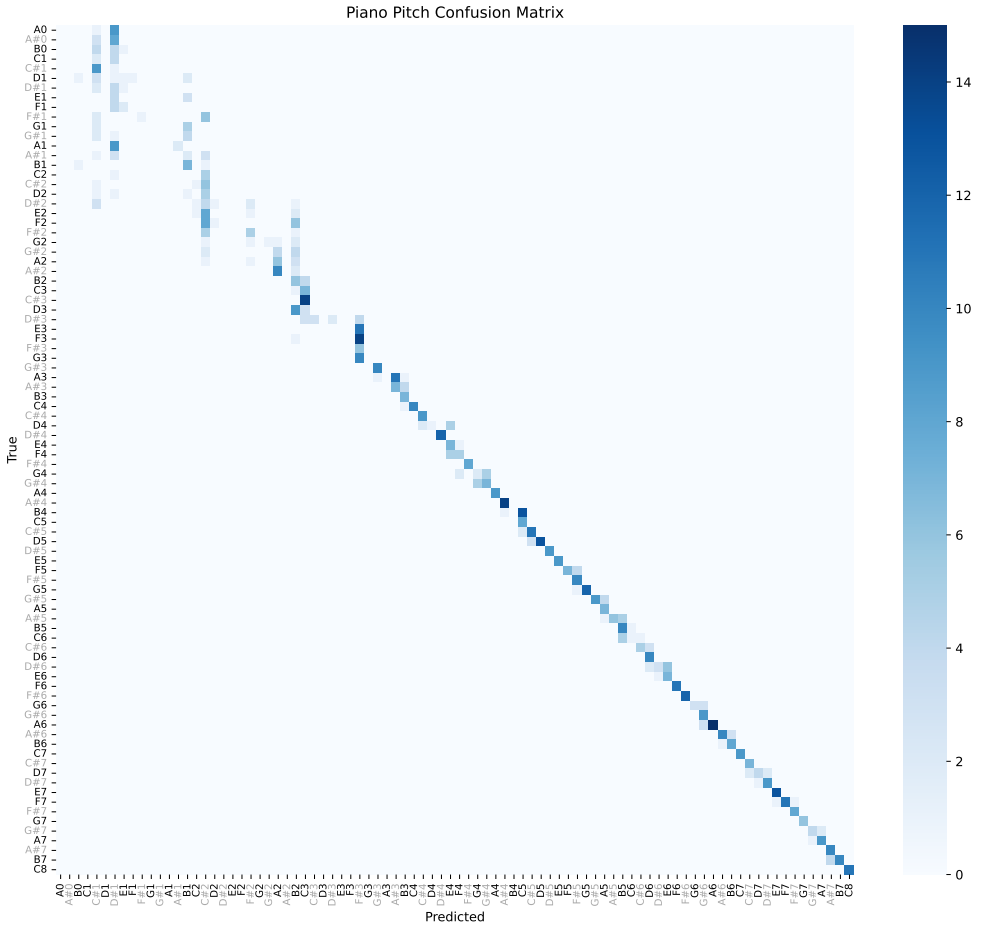
88-Key Piano Confusion Matrix Analysis: Error Patterns

Handy Preprocessing



Misclassification starts at B2 (123.47 Hz)

Spikify Preprocessing



Misclassification starts at F3 (174.61 Hz)

Key Observation. *Low-frequency notes are harder to classify.* Handy preprocessing extends accurate classification lower, suggesting better temporal precision at low freqs.

STDP Learning Rule

Classical STDP (Spike-Timing-Dependent Plasticity)

$$\Delta w_{ij} = \begin{cases} A_+ e^{-\Delta t/\tau_+} & \text{if } \Delta t = t_j^{\text{post}} - t_i^{\text{pre}} > 0 \quad \text{(LTP)} \\ -A_- e^{\Delta t/\tau_-} & \text{if } \Delta t < 0 \quad \text{(LTD)} \end{cases} \tag{10}$$

STDP-WTA Architecture:

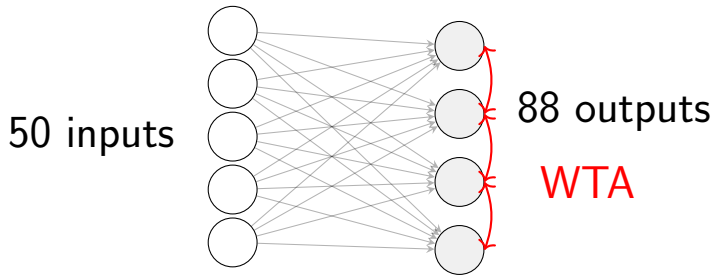
- **Input:** 50 cochlear channels $\Rightarrow$ **Output:** 88 LIF neurons (one per piano key)
- **Winner-Take-All (WTA):** Only highest-spiking neuron updates weights
- **Lateral inhibition:** $I_{\text{inh}} = 0.5 \cdot \sum_{k \neq j} s_k$

Trace-Based Implementation:

$$\text{pre\_trace}(t+1) = \text{pre\_trace}(t) \cdot e^{-1/\tau_{\text{pre}}} + s_i^{\text{pre}}(t) \tag{11}$$

$$\Delta w_{ij} = \eta \left(s_j^{\text{post}} \cdot \text{pre\_trace} - s_i^{\text{pre}} \cdot \text{post\_trace} \right) \tag{12}$$

STDP-WTA Architecture



Trace-Based Implementation:

$$\text{pre\_trace}(t+1) = \text{pre\_trace}(t) \cdot e^{-1/\tau_{\text{pre}}} + s_i^{\text{pre}}(t)$$

$$\text{post\_trace}(t+1) = \text{post\_trace}(t) \cdot e^{-1/\tau_{\text{post}}} + s_i^{\text{post}}(t)$$

$$\Delta w_{ij} = \eta \left(s_j^{\text{post}} \cdot \text{pre\_trace} - s_i^{\text{pre}} \cdot \text{post\_trace} \right)$$

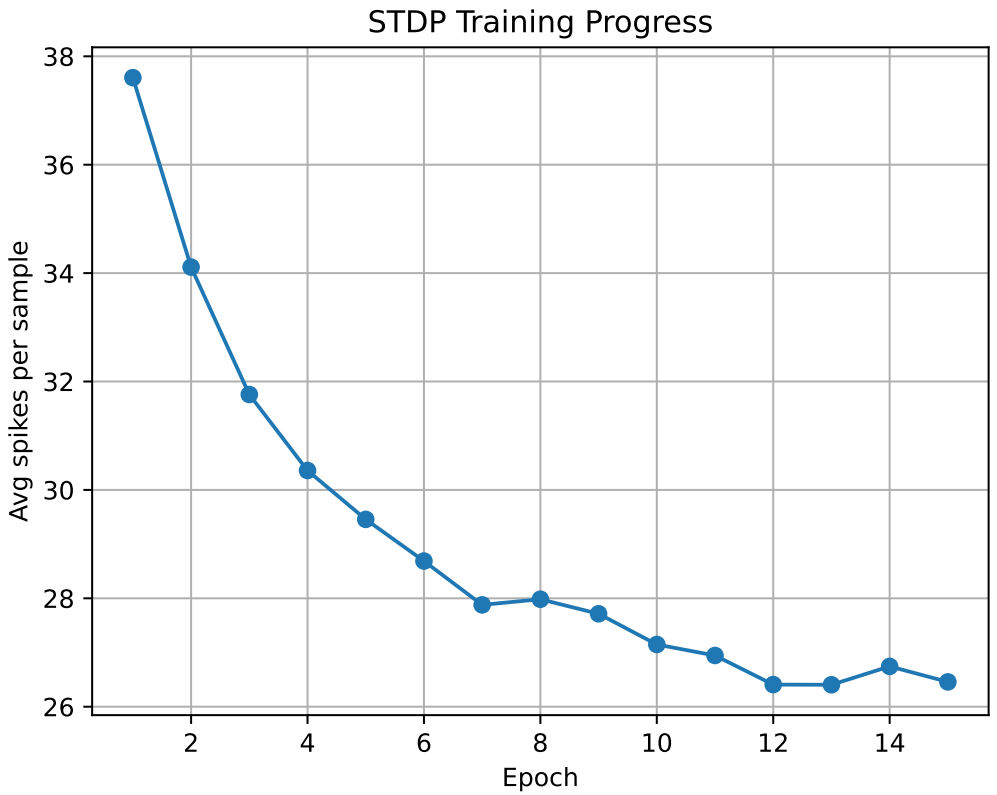
Training Procedure:

1. Forward pass to determine winner (max spike count)
2. Second pass with STDP update for winner only
3. Lateral inhibition prevents multiple winners
4. Homeostatic normalization after each update

Parameters: $\tau_{\text{pre}} = \tau_{\text{post}} = 20$ ms, $\eta = 0.002$, $\beta = 0.95$

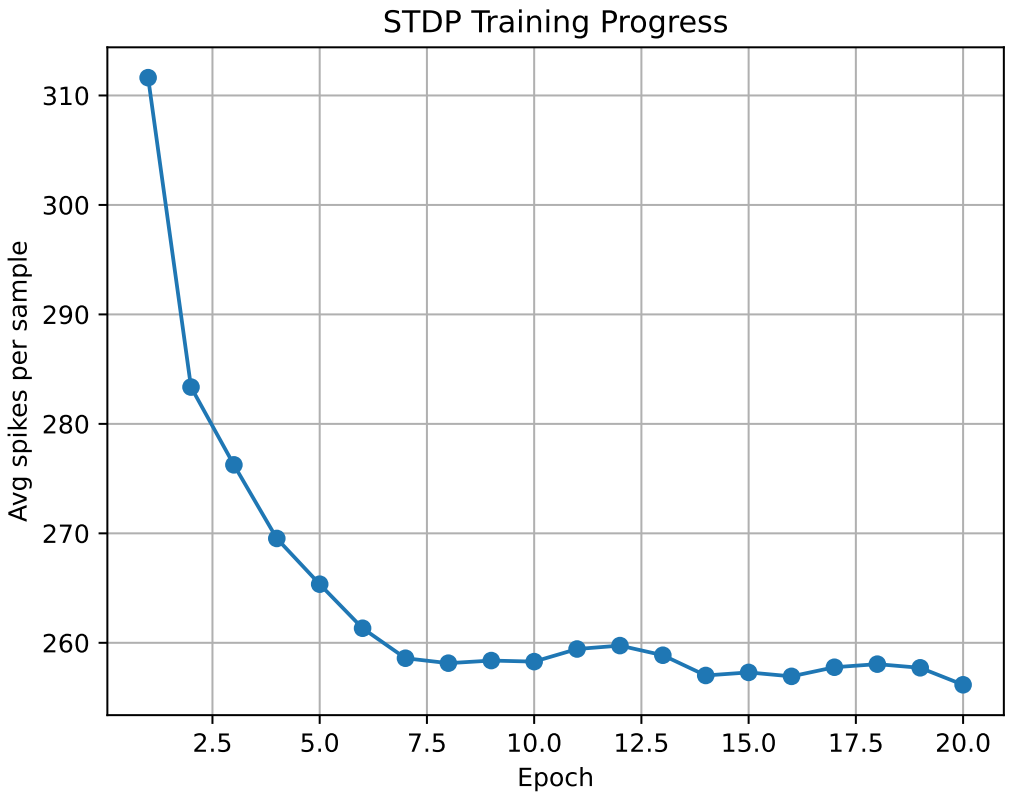
Spike Count Dynamics During Training

Handy Preprocessing



Gradual spike count reduction
indicates stable learning

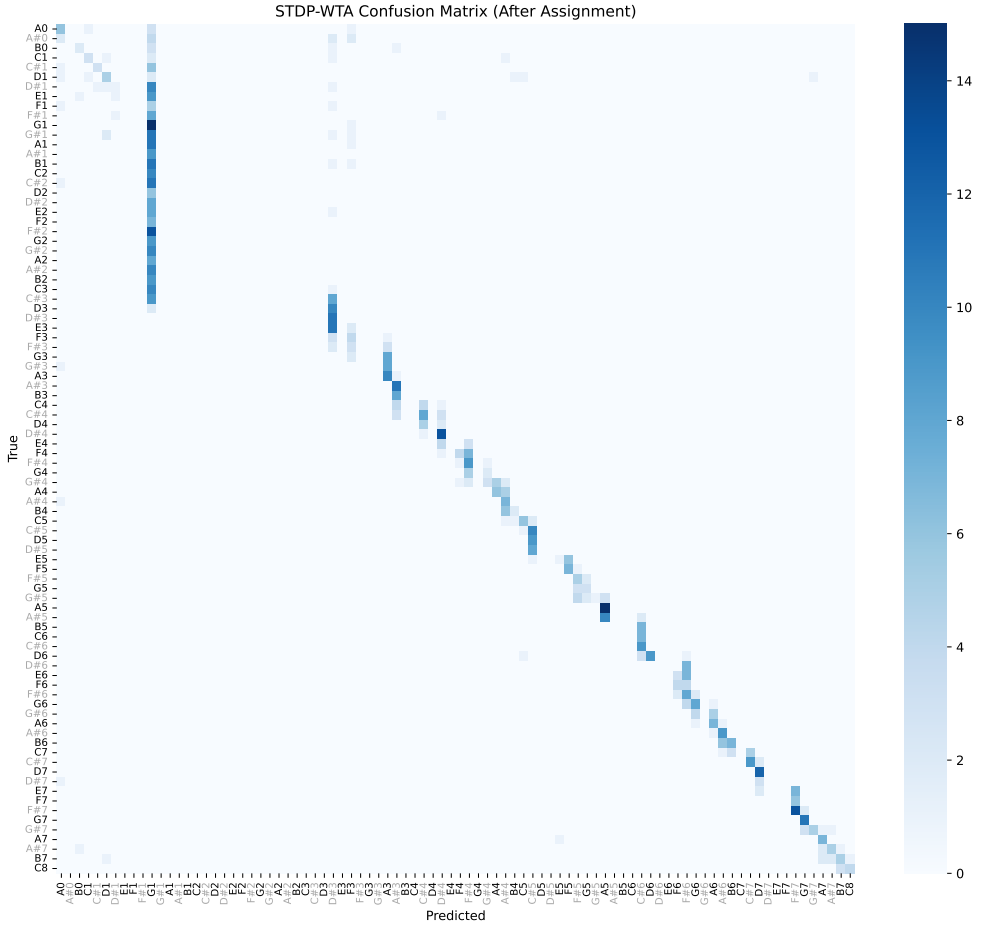
Spikify Preprocessing



Higher variance in spike counts
more unstable dynamics

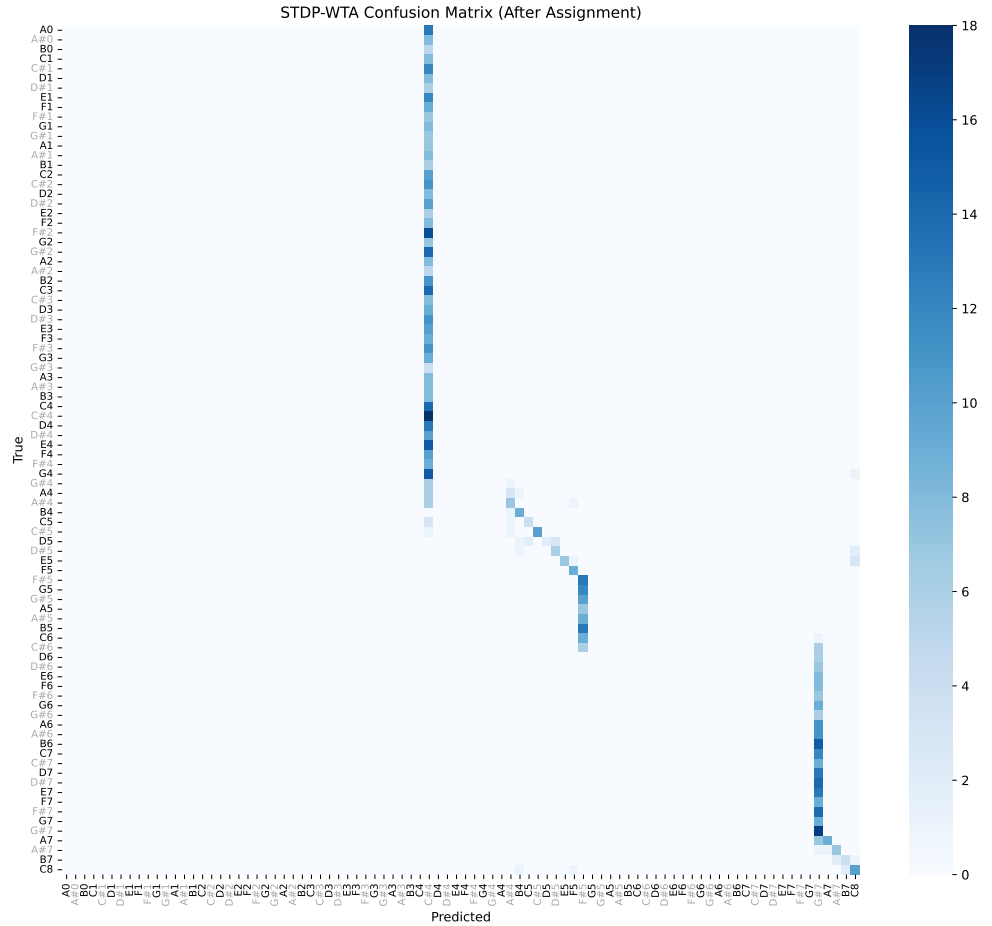
STDP Confusion Matrices: Learning Quality

Handy Preprocessing



Misclassifications (E3 and below) concentrated on G1/A1

Spikify Preprocessing



Diffuse misclassifications across frequency range

Comparison: Handy vs Spikify

Property	**Handy**	**Spikify**
Training speed	Slower	**Faster**
Noise robustness	**High** (stable)	Low (unstable)
Low-frequency accuracy	**B2 and above**	F3 and above
STDP convergence	**Stable**	Fluctuating
Error pattern	**Structured** (octave)	Diffuse
Temporal precision	**Better** phase-locking	Standard

Hypothesis: Handy's relative thresholding and global normalization preserve temporal structure better, enabling more robust phase-locking to stimulus frequency—critical for noise immunity.

Trade-off: Spikify's optimized implementation prioritizes training efficiency over noise robustness.

Conclusions. Current Limitation and Biological Inspiration

Current Approach Limitation

Training on all 88 keys simultaneously requires the model to learn:

1. Absolute frequency discrimination (within-octave)
2. Octave equivalence (C4 vs C5)
3. Full harmonic structure across 7+ octaves

This is computationally expensive. Possibly biologically implausible?

Human Absolute Pitch Development:

- Early childhood exposure to pitch-class associations
- Learning within limited frequency range initially
- Generalization to full frequency range later
- Octave equivalence emerges from harmonic structure

Future Research: Train on single octave (12 notes), then test generalization.

Proposed "Absolute Pitch" Architecture

1. Single-Octave Training (C4–B4)

- 12 classes instead of 88
- Learn pitch-class representation
- Octave-invariant feature extraction

2. Octave Generalization

- Transfer learned pitch-class representations
- Add octave detection layer
- Fine-tune on full range

Octave-Invariant Features:

$$f_{\text{normalized}} = f / 2^{\lfloor \log_2(f/f_{\text{ref}}) \rfloor} \tag{13}$$

Potential Benefits:

- Reduced parameter space (12 vs 88 output neurons)
- Better generalization through octave equivalence
- More biologically plausible learning trajectory?

Summary of Contributions

1. **Preprocessing Pipeline:**
   - Implemented custom ERB scales (Linear, Quadratic, Voicebox)
   - Discovered and fixed critical scaling bug
   - Established relative-threshold Poisson encoding
2. **3-Tone Classification:**
   - Achieved 100% accuracy after preprocessing fix
   - Demonstrated superior noise robustness of handy preprocessing
3. **88-Key Piano Classification:**
   - 58% accuracy on full piano range
   - Identified low-frequency classification challenges
4. **STDP Unsupervised Learning:**
   - Implemented competitive WTA learning
   - Showed structured learning with handy preprocessing

Future Directions

Immediate Improvements:

- Implement "absolute pitch" model with octave training

Research Questions:

- Can octave-invariant features improve generalization?
- How does filterbank density affect low-frequency performance?
- Can STDP with structured inhibition improve note separation?

Biological Extensions:

- Model binaural cues for pitch in noise
- Explore place-temporal coding trade-offs

References

1. Pan, Z., et al. (2019). An efficient and perceptually motivated auditory neural encoding algorithm for SNNs. *IEEE TNNLS*.
2. Song, Z., et al. (2024). Spiking-LEAF: A Learnable Auditory front-end for Spiking Neural Networks. *IEEE TPAMI*.
3. Wu, J., et al. (2018). A spiking neural network framework for robust sound classification. *Frontiers in Neuroscience*.
4. Alsakkal, M.H., & Wijekoon, J. (2025). Spiketrum: An FPGA-based implementation of a neuromorphic cochlea. *IEEE TCAS*.
5. Saddler, M.R., et al. (2021). Deep neural network models reveal interplay of peripheral coding and stimulus statistics in pitch perception. *Nature Communications*.
6. Glasberg, B.R., & Moore, B.C.J. (1990). Derivation of auditory filter shapes from notched-noise data. *Hearing Research*.
7. Cariani, P. (1999). Temporal coding of periodicity pitch in the auditory system. *Neural Plasticity*.

Thank You For Your Attention!

Neuromorphic Computing Course Project

Skoltech, May 2026